

DEEP REINFORCEMENT LEARNING

Double DQN · Dueling DQN · Actor-Critic

Numericals & Mini-Applications —

Presented by Vikas Kumar Tiwari

Research Scholar · Department of Computer Science
Indian Institute of Information Technology Lucknow (IIITL)
C.G. City, Lucknow - 226002, Uttar Pradesh
sites.google.com/view/vktiware
RL Tutorial · Online · 10:15 AM

What we'll cover

Same rhythm for each method: idea → equation → numerical → code

1

Double DQN

Fixes DQN's over-optimistic Q-values by splitting action choice from action value.

2

Dueling DQN

Splits Q into "how good is this state" (V) and "how good is each action" (A).

3

Actor-Critic

A policy (actor) that acts + a value net (critic) that grades each step via TD error.

RECAP

From Q-learning to DQN

Everything today builds on one target value we regress toward.

Bellman / Q-learning target

$$Q(s,a) \leftarrow r + \gamma \cdot \max Q(s',a')$$

γ = discount (e.g. 0.9), r = reward

DQN target (uses a target net θ^-)

$$y = r + \gamma \cdot \max Q(s',a'; \theta^-)$$

Train online net θ to match y

The problem we'll fix first

- The $\max$ operator both PICKS the best action and TRUSTS its value — from the same noisy network.
- Any random over-estimate gets selected by the max, so errors don't cancel — they pile up.
- Result: Q-values drift too high ("overestimation bias") and learning becomes unstable.

The fix: decouple selection from evaluation

Choose the action with the online net; score it with the target net.

Vanilla DQN (biased)

$$y = r + \gamma \cdot Q(s', \operatorname{argmax} Q(s', a'; \theta^-) ; \theta^-)$$

Same net picks AND scores it : over-optimistic

Double DQN (corrected)

$$a^* = \operatorname{argmax} Q(s', a'; \theta) \quad (\text{online net picks})$$

$$y = r + \gamma \cdot Q(s', a^*; \theta^-) \quad (\text{target net scores})$$

One-line intuition

If the online net over-rates an action by luck, the independent target net usually won't agree — so the inflated value is not rubber-stamped. Selection and evaluation use different weights, so noise no longer conspires with the max.

DOUBLE DQN

Numerical — watch the bias shrink

$r = 1$, $\gamma = 0.9$, next state s' has two actions a_1 , a_2 .

Given Q-values at s'

Online θ : $a_1 = 3.0$ $a_2 = 2.0$

Target θ^- : $a_1 = 1.0$ $a_2 = 2.5$

Vanilla DQN

$$y = 1 + 0.9 \cdot \max(1.0, 2.5) = 1 + 0.9 \cdot 2.5 = 3.25$$

Double DQN

$$\begin{aligned} \text{pick } a^* &= \operatorname{argmax} \theta = a_1 \rightarrow \text{score on } \theta^- = 1.0 \\ y &= 1 + 0.9 \cdot 1.0 = 1.9 \end{aligned}$$

3.25

Vanilla DQN target



1.9

Double DQN target

Why? DQN trusts the max value 2.5, but the online net's actual pick (a_1) is worth only 1.0. Double DQN evaluates the pick $\rightarrow$ no inflation.

Tiny code — only the target line changes

PyTorch. 'online' = θ , 'target' = θ^- .

double_dqn.py

```
# pick action with ONLINE, value it with TARGET
next_a = online(next_s).argmax(1, keepdim=True)
next_q = target(next_s).gather(1, next_a)

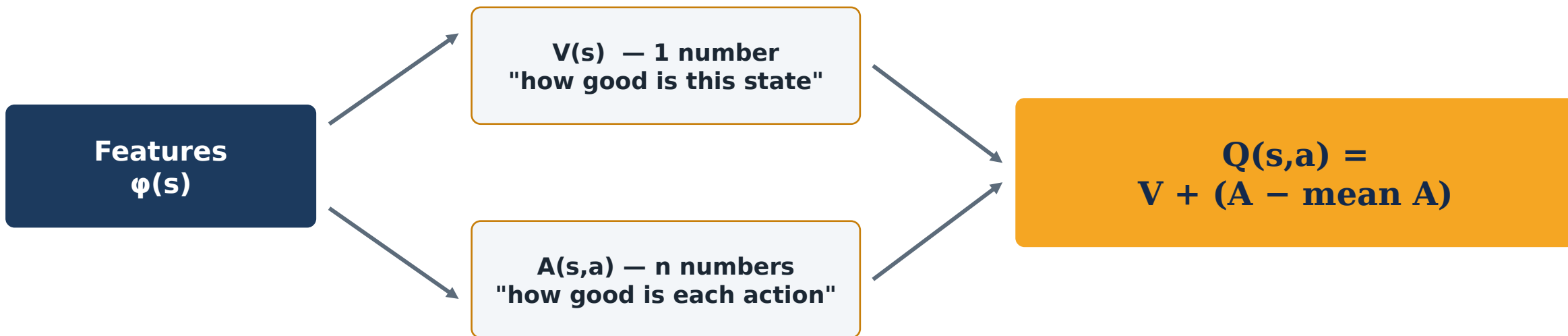
y = r + gamma * next_q * (1 - done)    # DDQN target
q = online(s).gather(1, a)
loss = F.mse_loss(q, y.detach())
loss.backward()
```

Reading it

- `argmax` on `online` → the action a^*
- `gather` on `target` → its value
- Plain DQN would call `max()` on `target` directly — that single change removes the bias.

Split Q into value + advantage

$$Q(s,a) = V(s) + (A(s,a) - \text{mean } A(s,a))$$



Why subtract the mean advantage? It stops V and A from drifting freely (only their sum, Q, is observed). Subtracting mean A pins the level to V and keeps the split identifiable.

Numerical — recombining the streams

$V(s) = 5.0$, advantages $A = [3, 1, 2]$ for a_1, a_2, a_3 .

Step 1 — mean advantage

$$\text{mean } A = (3 + 1 + 2) / 3 = 2$$

Step 2 — $Q = V + (A - \text{mean } A)$:

$$Q(a_1) = 5 + (3-2) = 6$$

$$Q(a_2) = 5 + (1-2) = 4$$

$$Q(a_3) = 5 + (2-2) = 5$$

6 $Q(a_1) \leftarrow \text{best}$ **5** $Q(a_3)$ **4** $Q(a_2)$

Note: shift every advantage by a constant and Q is unchanged — that's the mean-subtraction doing its job.

Best action = a_1 ($Q = 6$). The critic-like $V(s)=5$ sets the baseline; advantages just rank the actions around it.

Tiny code — one body, two heads

Reuse it anywhere a normal DQN net goes (incl. Double DQN).

dueling_dqn.py

```
class DuelingDQN(nn.Module):
    def __init__(self, n_in, n_act):
        super().__init__()
        self.body = nn.Linear(n_in, 128)
        self.V = nn.Linear(128, 1)      # value
        self.A = nn.Linear(128, n_act)  # advantage
    def forward(self, x):
        h = F.relu(self.body(x))
        V, A = self.V(h), self.A(h)
        return V + (A - A.mean(1, keepdim=True))
```

Reading it

- Shared body learns features once.
- V head → 1 output; A head → n outputs.
- Last line = the recombination formula, exactly as in the numerical.

ACTOR-CRITIC

Two networks that help each other

Actor acts; Critic grades each step with a TD error δ .



δ updates BOTH (actor $\uparrow$ good actions, critic $\downarrow$ its own error)

TD error is the shared learning signal

$\delta = r + \gamma \cdot V(s') - V(s)$ if $\delta > 0$, the action beat expectations, so do it more often

Numerical — one update step

Critic says $V(s)=2.0$, $V(s')=3.0$. Reward $r=1.0$, $\gamma=0.9$, $\alpha=0.1$.

Compute the TD error

$$\begin{aligned}\delta &= r + \gamma \cdot V(s') - V(s) \\ &= 1 + 0.9 \cdot 3.0 - 2.0 \\ &= 1 + 2.7 - 2.0 = 1.7\end{aligned}$$

Critic: $V(s) \leftarrow 2.0 + 0.1 \cdot 1.7 = 2.17$

Actor: $\uparrow \log \pi(a|s)$, scaled by $\delta = 1.7$

+1.7

TD error δ (good step)

2.17

new $V(s)$, toward 3.7

$\delta > 0 \Rightarrow$ the outcome beat the critic's prediction $\Rightarrow$ nudge $V(s)$ up AND make that action more likely.

Sign of δ is everything: positive $\rightarrow$ reinforce the action; negative $\rightarrow$ discourage it. Magnitude sets the step size.

Tiny code — one step, two losses

actor → probabilities, *critic* → $V(s)$.

actor_critic.py

```
probs = actor(s)                #  $\pi(a|s)$ 
v, v_next = critic(s), critic(s_next)

delta = r + gamma * v_next - v  # TD error  $\delta$ 

actor_loss = -torch.log(probs[a]) * delta.detach()
critic_loss = delta.pow(2)
(actor_loss + critic_loss).backward()
```

Reading it

- detach() on δ for the actor: don't push critic error through the policy.
- Critic minimises δ^2 (makes predictions accurate).
- Actor maximises log-prob weighted by δ (reinforces good actions).

SUMMARY

One-line cheat sheet

Method	Changes what?	Fixes / gives you	Key formula
Double DQN	the target value	less over-estimation	$y = r + \gamma \cdot Q(s', \arg\max \theta^- ; \theta^-)$
Dueling DQN	the network shape	faster value learning	$Q = V + (A - \text{mean } A)$
Actor-Critic	adds a policy net	direct, low-variance policy	$\delta = r + \gamma V(s') - V(s)$

All three still learn from the same reward signal — they just make the estimate less biased, more efficient, or directly policy-based.

Key takeaways

Double DQN

Pick with online, score with target → kills the max-bias. 3.25 → 1.9.

Dueling DQN

$Q = V + (A - \text{mean } A)$. Learn the state's worth once, rank actions around it.

Actor-Critic

TD error δ trains both nets. Sign of δ reinforces or discourages the action.

Thank you · Questions?

Vikas Kumar Tiwari · Research Scholar, CSE
IIIT Lucknow · sites.google.com/view/vktiware